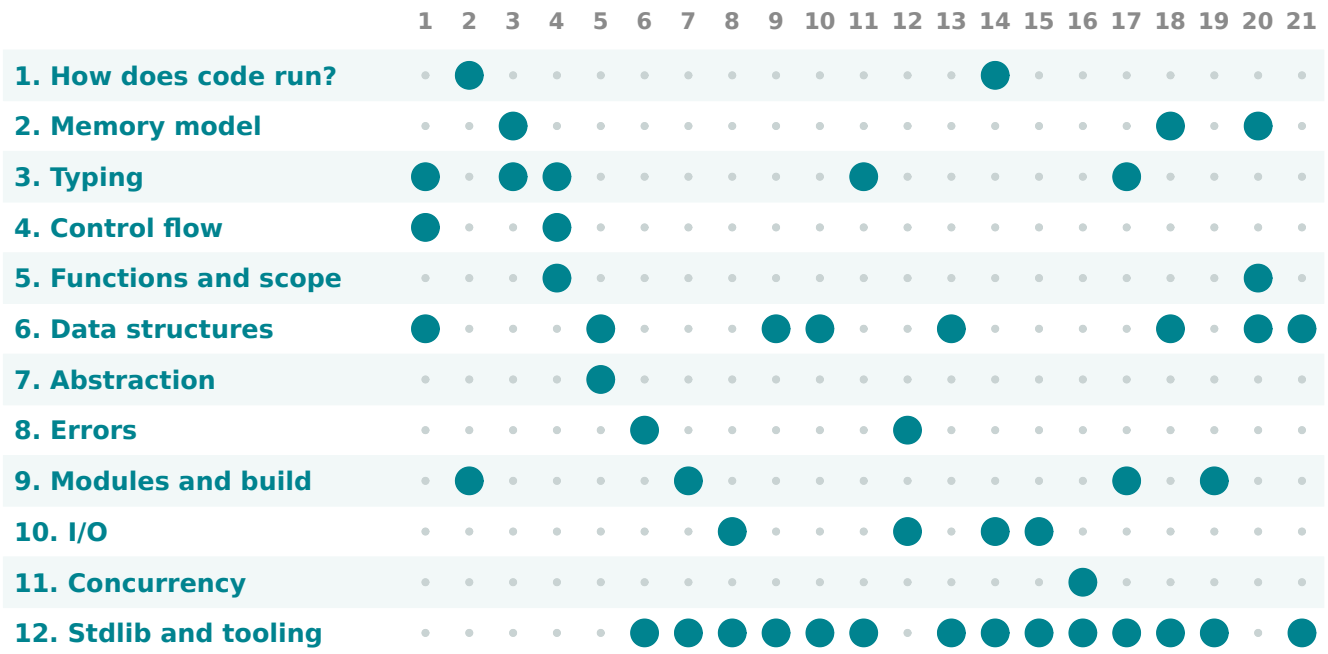
Appendix B

The language passport

the same twelve questions, for every language you learn

Each chapter ended with a stamp: Python's answer to the passport questions it covered. Here they are collected, first as a map of **where each answer was built**, then as one page. The last pages ask the same questions of C, Java and Rust, and give a ten-minute experiment for answering each one in any language.



Each dot is a chapter whose stamp answered that question. Data structures (question 6) and the stdlib (12) are stamped most because almost every chapter adds to them; abstraction and concurrency are answered in one place, so they are worth rereading before an exam.

> **KEY IDEA**
> The passport is a reading guide, not a summary. When a new language's tutorial says something, ask which of the twelve questions it answers. The parts it skips are where your Python habits will mislead you.

Python's passport

LANGUAGE PASSPORT: Python

same 12 questions for C, Java, Rust later

1. How does code run?

CPython compiles each module to bytecode (cached in `__pycache__`) and a stack VM runs it. Syntax errors at compile time, the rest at run time. To the OS: a process with argv, environment, three streams and an exit code (Ch 2, 14).

2. Memory model

Every value is a heap object; names, list slots and attributes hold references, and assignment never copies. Reference counting plus a cycle collector free memory. NumPy: one typed buffer, views share it (Ch 3, 18, 20).

3. Typing

Dynamic and strong: values have types, names don't; `'5' + 5` is a TypeError. Mutable vs immutable; only hashable objects are dict keys. Hints are checked by mypy or pyright, never by Python (Ch 3, 11, 17).

4. Control flow

Indentation makes blocks. `if`/`elif`, `for` over any iterable, `while`, `match`, loop `else`. Generators pause at `yield`; exceptions are non-local exits (Ch 1, 4, 6).

5. Functions and scope

First-class functions made when `def` runs; positional-only, keyword-only, `*args`, `**kwargs`; defaults evaluated once. LEGB scope, closures, decorators. No tail calls; recursion limit 1000 (Ch 4, 20).

6. Data structures

`list`, `tuple`, `dict` (ordered), `set`, `str`, `bytes` built in; `deque`, `Counter`, `heapq`, `bisect` in the stdlib. List end O(1), front O(n); dict and set O(1) on average (Ch 10, 20, 21).

7. Abstraction

Classes with multiple inheritance (C3 MRO, `super()`); duck typing through dunder protocols; ABCs and `typing.Protocol` for explicit interfaces; dataclasses, properties, enums (Ch 5).

8. Errors

Exceptions unwind to the first matching `except`; `else`, `finally`, `raise ... from`, `ExceptionGroup`. EAFP style; `with` guarantees clean-up; library errors form class trees (Ch 6, 12).

9. Modules and build

A module runs once, on first import, and is cached in `sys.modules`; found via `sys.path`. Packages are folders. A `venv` per project; pip installs wheels; `pyproject.toml` plus a lock file (Ch 2, 17, 19).

10. I/O

Files are bytes; text is decoded with an explicit encoding. `open` in a `with`, `pathlib`, many formats in the stdlib, `sqlite3` for structured data; sockets speak bytes (Ch 8, 12, 15).

11. Concurrency

Threads share memory (locks, queues), but the GIL blocks CPU parallelism on the default build: processes for CPU work, `asyncio` for many waits in one thread. Free-threaded builds since 3.13 (Ch 16).

12. Stdlib and tooling

Batteries included: 195 public modules (Appendix A). `unittest`, `doctest`, `pdb`, `timeit`, `cProfile` built in; pip, mypy and Git outside (Ch 7, 17, 19).

Four passports side by side

The same questions for the languages a CS degree usually meets next. One line each is a simplification; it tells you where to look, not everything you'll find.

	Python	C	Java	Rust
1. How does code run?	bytecode on the CPython VM	compiled ahead of time to machine code, then linked	compiled to JVM bytecode, JIT-compiled while running	compiled ahead of time to machine code by rustc
2. Memory model	all values are heap objects; refcount + cycle GC	you manage it: stack values, `malloc`/`free`, pointers	primitives by value, objects by reference; tracing GC	ownership and borrowing, checked at compile time; no GC
3. Typing	dynamic, strong; optional hints	static, weak (implicit conversions, casts)	static, strong, nominal; generics by erasure	static, strong, inferred; traits; no implicit conversions
4. Control flow	indentation; `for` over iterables; `match`	braces; counting `for` loops; `switch`; `goto`	braces; enhanced `for`; `switch` expressions	everything is an expression; exhaustive `match`
5. Functions and scope	first-class, closures, LEGB, `*args`	function pointers, no closures; pass by value	methods in classes; lambdas capture effectively-final	functions and closures (`Fn`, `FnMut`, `FnOnce`); move or borrow
6. Data structures	`list`, `dict`, `set` built in; `deque`, `heapq`	arrays and structs; build the rest yourself	Collections: `ArrayList`, `HashMap`, `TreeMap`, `ArrayDeque`	`Vec`, `HashMap`, `BTreeMap`, `VecDeque`, `BinaryHeap`
7. Abstraction	classes, multiple inheritance, duck typing	structs + function pointers; headers as interfaces	classes, single inheritance, interfaces	structs and enums + traits; no inheritance
8. Errors	exceptions; `with` for clean-up	return codes and `errno`	exceptions (checked and unchecked); try-with-resources	`Result<T, E>`, `Option`, the `?` operator; panic for bugs
9. Modules and build	`import`, venv + pip, `pyproject.toml`	`#include`, separate compilation, make/CMake	packages and JARs; Maven or Gradle	crates and modules; Cargo and crates.io
10. I/O	`open()` file objects; text vs bytes	`FILE*` from stdio, or POSIX `read`/`write`	`java.io` streams; `Files` and `Path` (java.nio)	`std::fs`, `Read`/`Write` traits returning `Result`
11. Concurrency	threads (GIL), processes, `asyncio`	pthreads, `fork`, C11 atomics	threads, executors and concurrent collections, virtual threads	threads with `Send`/`Sync` checked by the compiler; async runtimes
12. Stdlib and tooling	huge stdlib; pip, unittest, pdb	small libc; gdb, valgrind, sanitizers	large JDK; jshell, JUnit, Flight Recorder	lean std; cargo test, fmt, clippy, doc

Ten-minute probes

Run these in any new language, in order. Each one forces an answer to one passport question, and the surprises are what to write in that language's passport.

question	the experiment
1. How does code run?	Write hello world and run it. Which files appear? Add a type error: is it reported before running or only when that line runs?
2. Memory model	Make two names for one list, change it through one, print the other. Then pass it to a function that changes it.
3. Typing	Try `"5" + 5`, `5 / 2` and storing text in a number variable. Which fail, and when?
4. Control flow	Loop over a list, a map and a string. Is there a `match`/`switch`, and must it cover every case?
5. Functions and scope	Return a function that counts its calls (a closure). Can it change the captured variable?
6. Data structures	Find the growable array, hash map, set, deque and heap. Look up the cost of inserting at the front.
7. Abstraction	Define a Shape with `area()` and two implementations; call `area()` on a list holding both.
8. Errors	Open a missing file. Exception, error value, or crash? How is clean-up guaranteed?
9. Modules and build	Split a program into two files, import one from the other, and add one third-party package.
10. I/O	Copy a UTF-8 file line by line and read stdin. What is the default encoding?
11. Concurrency	Add 1 to a shared counter from 4 threads, 100,000 times each. Is the total right?
12. Stdlib and tooling	Write and run one unit test, step through it in the debugger, and time a loop.

RULE OF THUMB

Pair the passport with the library card (Ch 7) for every library and the spike for every unknown: twelve questions for the language, six for each library, twenty lines of code for anything you're unsure of.